```
import pandas as pd

# 1. Import the dataset and inspect its structure
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master
df = pd.read_csv(url)

print("--- Dataset Information ---")
print(df.info())
print("\n--- First 5 Rows ---")
print(df.head())

# 2. Identify missing values and duplicate records
print("\n--- Missing Values Before Cleaning ---")
print(df.isnull().sum())

print("\n--- Duplicate Records Before Cleaning ---")
print(df.duplicated().sum())

# 3. Clean the dataset (handling null values and removing duplicates)
df_cleaned = df.dropna()
df_cleaned = df_cleaned.drop_duplicates()

# Verify the cleaning process
print("\n--- Missing Values After Cleaning ---")
print(df_cleaned.isnull().sum())

# 4. Save the cleaned dataset as a new CSV file
df_cleaned.to_csv('cleaned_dataset.csv', index=False)
print("\nProcess Completed: 'cleaned_dataset.csv' saved successfully.")
```

```
3      0            113803  53.1000  C123        S
4      0            373450   8.0500   NaN        S

--- Missing Values Before Cleaning ---
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64

--- Duplicate Records Before Cleaning ---
0

--- Missing Values After Cleaning ---
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin          0
Embarked       0
dtype: int64

Process Completed: 'cleaned_dataset.csv' saved successfully.
```

```
import pandas as pd

# 1. Load the cleaned dataset from Task 1
df_eda = pd.read_csv('cleaned_dataset.csv')

print("=== Task 2: Exploratory Data Analysis (EDA) ===\n")

# 2. Examine features using descriptive statistics
print("--- Summary Statistics (Numerical Data) ---")
print(df_eda.describe())

# 3. Identify trends and distributions
print("\n--- Survival Count (0 = No, 1 = Yes) ---")
print(df_eda['Survived'].value_counts())

print("\n--- Passenger Class Distribution (1st, 2nd, 3rd Class) ---")
```

```
print(df_eda['Pclass'].value_counts())

# 4. Use summary statistics to answer key questions (e.g., Average Fare
print("\n--- Average Ticket Fare by Passenger Class ---")
avg_fare = df_eda.groupby('Pclass')['Fare'].mean()
print(avg_fare)

print("\nProcess Completed: Exploratory Data Analysis successfully execu
```

```
=== Task 2: Exploratory Data Analysis (EDA) ===

--- Summary Statistics (Numerical Data) ---
       PassengerId    Survived      Pclass         Age       SibSp  \
count   183.000000  183.000000  183.000000  183.000000  183.000000
mean    455.366120    0.672131    1.191257   35.674426    0.464481
std     247.052476    0.470725    0.515187   15.643866    0.644159
min       2.000000    0.000000    1.000000    0.920000    0.000000
25%     263.500000    0.000000    1.000000   24.000000    0.000000
50%     457.000000    1.000000    1.000000   36.000000    0.000000
75%     676.000000    1.000000    1.000000   47.500000    1.000000
max     890.000000    1.000000    3.000000   80.000000    3.000000

            Parch        Fare
count  183.000000  183.000000
mean     0.475410   78.682469
std      0.754617   76.347843
min      0.000000    0.000000
25%      0.000000   29.700000
50%      0.000000   57.000000
75%      1.000000   90.000000
max      4.000000  512.329200

--- Survival Count (0 = No, 1 = Yes) ---
Survived
1    123
0     60
Name: count, dtype: int64

--- Passenger Class Distribution (1st, 2nd, 3rd Class) ---
Pclass
1    158
2     15
3     10
Name: count, dtype: int64

--- Average Ticket Fare by Passenger Class ---
Pclass
1    88.683228
2    18.444447
3    11.027500
Name: Fare, dtype: float64

Process Completed: Exploratory Data Analysis successfully executed.
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Load the dataset
df_viz = pd.read_csv('cleaned_dataset.csv')
print("=== Task 3: Data Visualization Dashboard ===\n
print("Generating charts, please wait...")

# Set the background style for the charts
sns.set_theme(style="whitegrid")

# Create a figure to hold 4 different charts
plt.figure(figsize=(12, 10))

# Chart 1: Bar Chart - Survival Count
plt.subplot(2, 2, 1)
sns.countplot(x='Survived', data=df_viz, palette='Se
plt.title('Survival Count (0 = No, 1 = Yes)')
plt.xlabel('Survived Status')
plt.ylabel('Total Count')

# Chart 2: Pie Chart - Passenger Class Distribution
plt.subplot(2, 2, 2)
class_counts = df_viz['Pclass'].value_counts()
plt.pie(class_counts, labels=['1st Class', '2nd Clas
plt.title('Passenger Class Distribution')

# Chart 3: Histogram - Age Distribution
plt.subplot(2, 2, 3)
sns.histplot(df_viz['Age'], bins=20, kde=True, color
plt.title('Age Distribution of Passengers')
plt.xlabel('Age in Years')
plt.ylabel('Frequency')

# Chart 4: Scatter Plot - Age vs Fare
plt.subplot(2, 2, 4)
sns.scatterplot(x='Age', y='Fare', hue='Survived', da
plt.title('Age vs Ticket Fare (by Survival)')
plt.xlabel('Age')
plt.ylabel('Ticket Fare')

# Adjust layout so charts don't overlap and display
plt.tight_layout()
plt.show()

print("\nProcess Completed: Data visualizations gener
```

```
=== Task 3: Data Visualization Dashboard ===

Generating charts, please wait...
/tmp/ipykernel_2508/3925040504.py:18: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be remov

  sns.countplot(x='Survived', data=df_viz, palette='Set2')
```